

Where Am I? Position

How a model knows the order of words · video 2:26

Embeddings tell the model **what** each token is, but not **where** it is: “The cat sat on the mat” and “The mat sat on the cat” get exactly the same vectors. This lesson is about putting the **order into the vectors themselves**: first two simple ideas, then the sine waves of the original transformer, and finally the rotations (RoPE) used by most modern models.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 10).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Same tokens, different meaning	▶ 0:08-0:31	3	☐
2. Idea 1: add the position number	▶ 0:31-0:43	3	☐
3. Idea 2: learn a vector per position	▶ 0:43-0:58	4	☐
4. Waves at many speeds, like a clock	▶ 0:58-1:29	3	☐
5. Position is added to meaning	▶ 1:29-1:41	3	☐
6. Sinusoidal encoding in code	▶ 1:41-1:54	3	☐
7. RoPE: rotate instead of add	▶ 1:54-2:14	4	☐

Before you start: this lesson builds on episode 3 (embeddings: one vector per token, $x = E[\text{ids}]$). The maths is adding vectors, sine and cosine waves, and rotation: Foundations F01 (Vectors) and F09 (Waves and Rotations) cover it. Concept 6 reads a short NumPy function; F11 (NumPy in Three Minutes) helps.

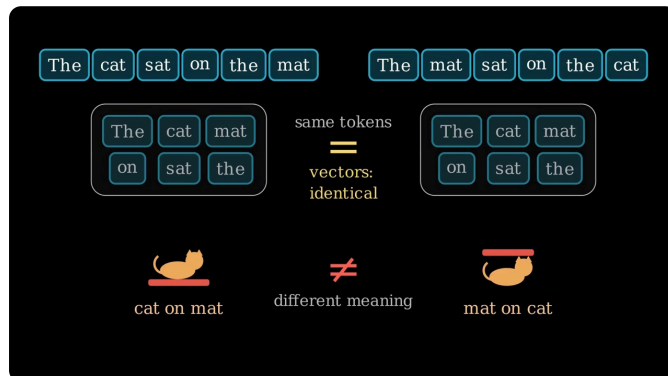
CONCEPT 1 OF 7

Same tokens, different meaning

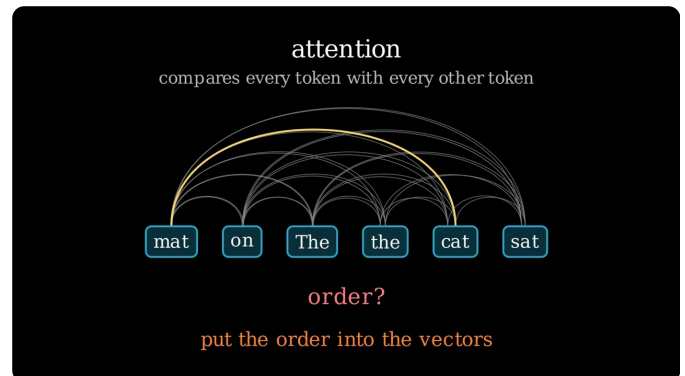
▶ Video 0:08-0:31

“The cat sat on the mat” and “The mat sat on the cat” contain the same tokens but mean completely different things. So far, our token vectors can't tell them apart: a token gets the same embedding wherever it appears.

And it matters. **Attention**, the heart of the transformer (next episode), compares every token with every other token, and nothing in that comparison says which one came first. So we have to **put the order into the vectors themselves**.



▶ 0:16 Same tokens, different meaning, but identical vectors.



▶ 0:30 Attention compares every token with every other; nothing says which came first.

KEY IDEA

Without position information, a sentence is just a **bag of tokens**: word order is invisible. Each token vector must also carry **where** the token is.

Check 1: Same tokens, different meaning

Answer all 3 questions before looking at the answers. Pass mark: all correct.

1.1 TRUE OR FALSE

“Embeddings alone already tell “The cat sat on the mat” apart from “The mat sat on the cat”.”

True ☐ False ☐

1.2 MULTIPLE CHOICE

Which pair of sentences would look identical to a model with no position information?

- A. The dog bit the man / The man bit the dog
- B. The dog bit the man / A dog bit a man
- C. The dog bit / The dog bites
- D. I like tea / I like coffee

1.3 SHORT ANSWER

Why can't attention work out the word order by itself?

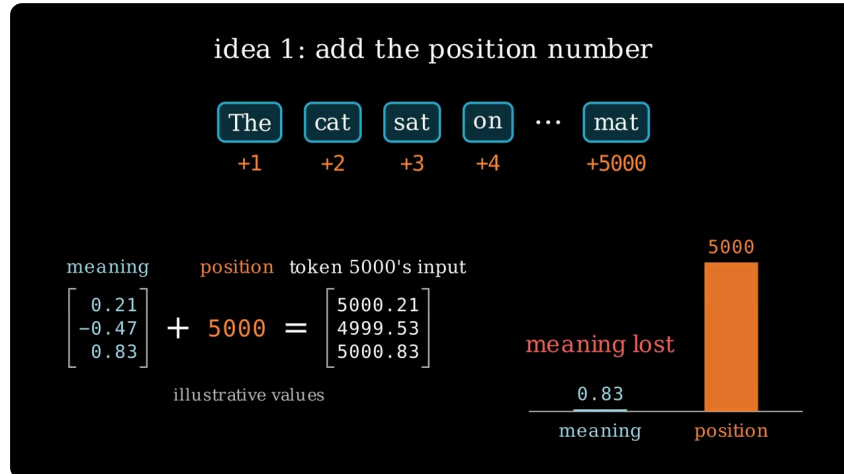
.....

CONCEPT 2 OF 7

Idea 1: add the position number

▶ Video 0:31-0:43

The most obvious idea: just add the position number to the token's vector: 1 for the first token, 2 for the second, and so on. But these numbers **grow without limit**. By token 5000, a meaning of (0.21, -0.47, 0.83) becomes (5000.21, 4999.53, 5000.83): the position **drowns out the meaning**.



▶ 0:42 At token 5000, the position (5000) drowns out the meaning (0.83) (illustrative values).

KEY IDEA

A position signal must **stay small**, on the same scale as the meaning. Raw position numbers grow without limit and swamp it.

Check 2: Idea 1: add the position number

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

The meaning vector is (0.21, -0.47, 0.83). With idea 1, what is the input vector at position 3?

2.2 MULTIPLE CHOICE

What goes wrong with idea 1 in a long text?

- A. Two positions get the same number
- B. The position numbers become huge and drown out the meaning
- C. It needs a table with one row per position
- D. It only works for even positions

2.3 TRUE OR FALSE

"At position 5000, cat (0.21, -0.47, 0.83) and a word with meaning (0.25, -0.40, 0.80) give input vectors that are almost identical."

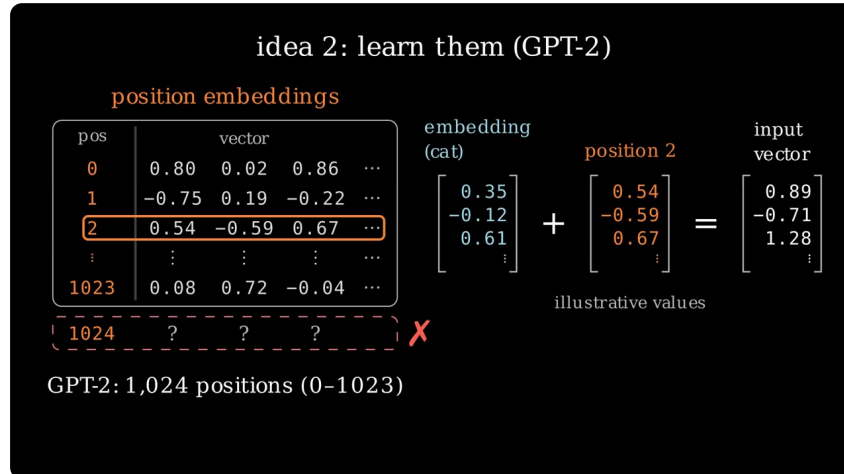
True ☐ False ☐

CONCEPT 3 OF 7

Idea 2: learn a vector per position

▶ Video 0:43–0:58

GPT-2 **learns** positions. It keeps a second table, the **position embeddings**, with one vector for each position, and adds that vector to the token's embedding. For *cat* at position 2: $(0.35, -0.12, 0.61, \dots) + (0.54, -0.59, 0.67, \dots) = (0.89, -0.71, 1.28, \dots)$.



▶ 0:57 Row 2 of the position table is added to cat's embedding.

KEY IDEA

Learned positions: **input = token embedding + position embedding**. Simple, but the table has a fixed size (GPT-2: 1,024 positions, 0–1023), and the model can't handle positions it never saw in training.

Check 3: Idea 2: learn a vector per position

Answer all 4 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

Using the table in the figure, compute the first three numbers of *cat* at position 0. (cat's embedding starts 0.35, -0.12, 0.61.)

3.2 MULTIPLE CHOICE

GPT-2 is given a text of 1,500 tokens. What is the problem?

- A. Tokens after position 1023 have no position vector
- B. The position numbers get too big
- C. Its vocabulary is too small
- D. There is no problem

3.3 CALCULATION

GPT-2's position table has 1,024 rows of 768 numbers. How many numbers must be learned?

3.4 TRUE OR FALSE

"In GPT-2, the position vectors are computed from a fixed formula."

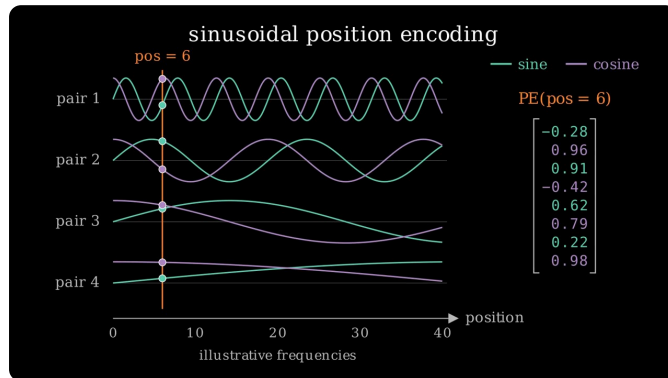
True ☐ False ☐

CONCEPT 4 OF 7

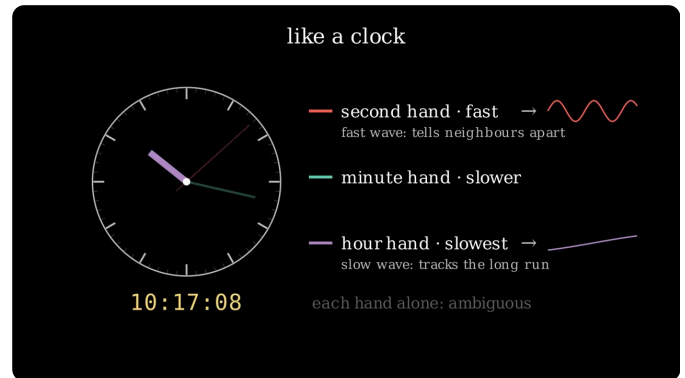
Waves at many speeds, like a clock

▶ Video 0:58-1:29

The original transformer paper used **waves**. Dimensions come in pairs: each pair follows a **sine** and a **cosine** wave, and every pair has its **own frequency**. To encode a position, read off where each wave is at that point. Sines and cosines always stay between -1 and 1 , so unlike idea 1 the values never grow.



▶ 1:12 Each pair: a sine and a cosine at its own frequency. Read off the waves at a position (here 6).



▶ 1:28 Like a clock: fast hands tell neighbours apart, slow hands track the long run.

It works like a clock. The second hand moves fast, the minute hand slower, the hour hand slowest. Each hand alone is ambiguous, but read together they give the exact time. Likewise, **fast waves tell neighbours apart**, and **slow waves track where you are in the long run**.

KEY IDEA

Sinusoidal encoding: position $\rightarrow$ sines and cosines at many frequencies. Every value is between -1 and 1 , and all the waves together pin down the exact position.

Check 4: Waves at many speeds, like a clock

Answer all 3 questions before looking at the answers. Pass mark: all correct.

4.1 MULTIPLE CHOICE

Why is one fast wave not enough to encode position?

- A. Its values are too large
- B. It repeats quickly, so far-apart positions can look the same
- C. It can't tell neighbours apart
- D. It needs a learned table

4.2 SHORT ANSWER

A clock's second hand points at 12 and its minute hand at 3. Give two different times this could be. Which hand settles it?

4.3 TRUE OR FALSE

"In sinusoidal encoding, the values for position 5000 are much larger than for position 5."

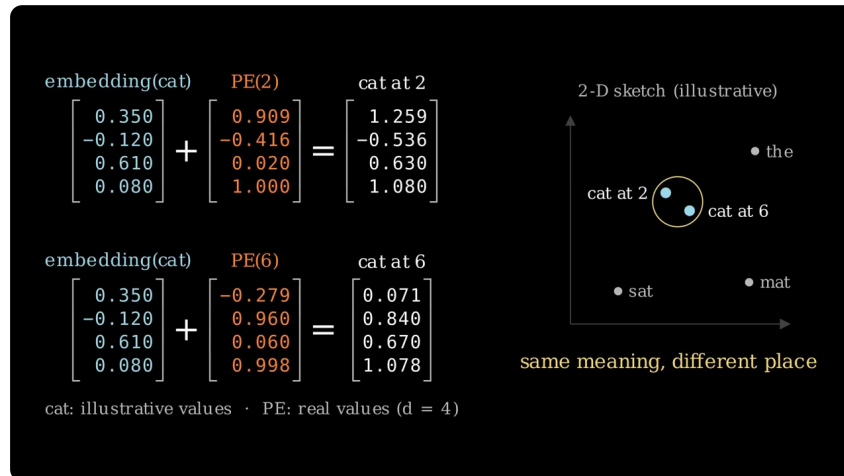
True ☐ False ☐

CONCEPT 5 OF 7

Position is added to meaning

▶ Video 1:29-1:41

The position vector is simply **added** to the token's embedding. With $d = 4$ numbers per token, the real sinusoidal values are $PE(2) = (0.909, -0.416, 0.020, 1.000)$ and $PE(6) = (-0.279, 0.960, 0.060, 0.998)$. So *cat* at position 2 and *cat* at position 6 start out as different vectors: the same meaning, in a different place.



▶ 1:40 cat at 2 and cat at 6: the same meaning, in a different place. PE values are real ($d = 4$).

KEY IDEA

input vector = embedding(token) + PE(position): one vector that carries both what the token is and where it is.

Check 5: Position is added to meaning

Answer all 3 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

For $d = 4$, $PE(0) = (\sin 0, \cos 0, \sin 0, \cos 0)$. Write $PE(0)$, then compute *cat* at position 0, using *cat*'s embedding $(0.350, -0.120, 0.610, 0.080)$.

5.2 TRUE OR FALSE

"Adding position vectors changes the rows of the embedding matrix **E**."

True ☐ False ☐

5.3 MULTIPLE CHOICE

cat appears at positions 2 and 6 of a sentence. Which statement is true?

- A.** Both get exactly the same input vector
- B.** Their input vectors differ, because different position vectors were added
- C.** Only the first *cat* gets a position vector
- D.** The second *cat* is removed

CONCEPT 6 OF 7

Sinusoidal encoding in code

▶ Video 1:41-1:54

In code: for each position and each pair of dimensions, compute an angle. Put the **sine in the even dimensions** and the **cosine in the odd ones**, then add the result to the embeddings.

```
1 def positional_encoding(n_pos, d_model):
2     pos = np.arange(n_pos)[:, None]      # 0, 1, 2, ...
3     i = np.arange(0, d_model, 2)[None, :] # even dimensions
4     angle = pos / 10000 ** (i / d_model)
5     pe = np.zeros((n_pos, d_model))
6     pe[:, 0::2] = np.sin(angle)          # even dims: sine
7     pe[:, 1::2] = np.cos(angle)          # odd dims: cosine
8     return pe
9
10 x = E[ids] + positional_encoding(len(ids), d_model)
```

▶ 1:53 The whole encoding in a few lines, added to the embeddings on the last line.

```
def positional_encoding(n_pos, d_model):
    pos = np.arange(n_pos)[:, None]      # 0, 1, 2, ...
    i = np.arange(0, d_model, 2)[None, :] # even dimensions
    angle = pos / 10000 ** (i / d_model)
    pe = np.zeros((n_pos, d_model))
    pe[:, 0::2] = np.sin(angle)          # even dims: sine
    pe[:, 1::2] = np.cos(angle)          # odd dims: cosine
    return pe

x = E[ids] + positional_encoding(len(ids), d_model)
```

KEY IDEA

$\text{angle} = \text{pos} / 10000 ** (i / d_model)$: the first pair ($i = 0$) uses the angle pos itself, the fastest wave; each later pair divides by a bigger number, so its wave is slower.

Check 6: Sinusoidal encoding in code

Answer all 3 questions before looking at the answers. Pass mark: all correct.

6.1 CALCULATION

With `d_model = 4`, the pairs use $i = 0$ and $i = 2$. What are the two angles at position 2?

6.2 CALCULATION

A sine wave repeats each time its angle grows by $2\pi \approx 6.28$. With `d_model = 4`, about how many positions does it take for pair 1 to repeat? And pair 2?

6.3 HANDS-ON CODE

Try it yourself. Copy `positional_encoding` from above (with `import numpy as np`) and run the lines below. (a) What are rows 0 and 2 of `pe`? (b) What is the largest absolute value anywhere in `big`, which covers 10,000 positions? (c) Compute the three dot products. What do you notice?

```
pe = positional_encoding(8, 4)
print(np.round(pe[0], 3), np.round(pe[2], 3))      # (a)

big = positional_encoding(10000, 16)
print(np.abs(big).max())                          # (b)

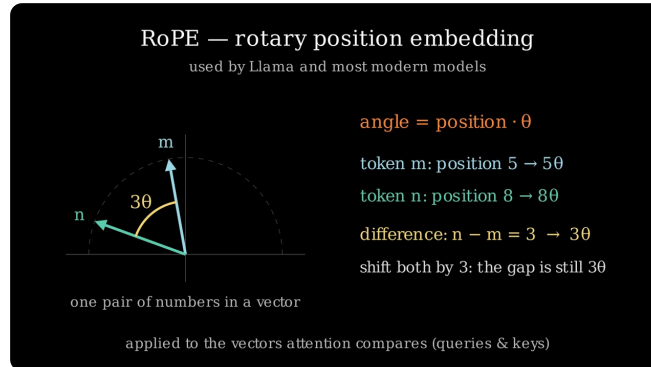
p = positional_encoding(50, 16)
print(p[10] @ p[11], p[30] @ p[31], p[10] @ p[40]) # (c)
```

CONCEPT 7 OF 7

RoPE: rotate instead of add

▶ Video 1:54-2:14

Most modern models, like Llama, use **rotary position embedding (RoPE)**. Instead of adding a vector, it **rotates** pairs of numbers by an angle that grows with position: $\text{angle} = \text{position} \cdot \theta$. A token at position 2 is turned by 2θ , one at position 5 by 5θ .



▶ 2:13 Shift both by 3: the gap stays 3θ .

RoPE is applied to the vectors attention compares (queries and keys, episode 6). There only the **difference** of the angles matters: $5\theta - 2\theta = 3\theta$, and at positions 5 and 8 it is still 3θ .

KEY IDEA

RoPE **rotates** pairs of numbers by position · θ instead of adding a vector. A comparison of two tokens then depends only on their **distance**.

Check 7: RoPE: rotate instead of add

Answer all 4 questions before looking at the answers. Pass mark: all correct.

7.1 CALCULATION

With RoPE, token A is at position 4 and token B at position 10. By what angles are they rotated, and what angle gap does the comparison see?

7.2 CALCULATION

Take $\theta = 10^\circ$. Two tokens sit at positions 3 and 7. Later in the text, the same two tokens sit at positions 103 and 107. What gap does the comparison see each time?

7.3 MULTIPLE CHOICE

How is RoPE different from sinusoidal encoding?

- A. It adds a larger position vector
- B. It rotates pairs of numbers by a position-dependent angle instead of adding a vector
- C. It uses a learned table, like GPT-2
- D. It ignores position

7.4 TRUE OR FALSE

“With RoPE, what a comparison sees depends on the tokens' exact positions, not just on how far apart they are.”

True ☐ False ☐

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Same tokens, different meaning · rewatch ▶ 0:08-0:31

1.1 False. Both sentences contain the same tokens, so they get exactly the same vectors.

1.2 A. Same tokens, different order. Every other pair differs in at least one token.

1.3 Nothing in its comparisons says which token came first. It compares every token with every other token; unless the vectors carry position, order is lost.

2. Idea 1: add the position number · rewatch ▶ 0:31-0:43

2.1 (3.21, 2.53, 3.83). Add 3 to every number: $-0.47 + 3 = 2.53$.

2.2 B. By token 5000 the numbers are around 5000, while the meaning is below 1.

2.3 True. (5000.21, 4999.53, 5000.83) vs (5000.25, 4999.60, 5000.80): the small differences in meaning are lost next to 5000.

3. Idea 2: learn a vector per position · rewatch ▶ 0:43-0:58

3.1 (1.15, -0.10, 1.47). Row 0 is (0.80, 0.02, 0.86): $0.35 + 0.80$, $-0.12 + 0.02$, $0.61 + 0.86$.

3.2 A. The table has 1,024 rows (positions 0-1023), all learned in training.

3.3 786,432. $1,024 \times 768$. Like the embedding matrix, every one of them is learned.

3.4 False. They are learned in training, just like the token embeddings. (The formula is the next idea.)

4. Waves at many speeds, like a clock · rewatch ▶ 0:58-1:29

4.1 B. Like the second hand, which is back in the same place every minute.

4.2 E.g. 1:15:00 or 4:15:00; the hour hand. The fast hands only fix the time within the hour. The slowest hand tells which hour.

4.3 False. Every value is a sine or cosine, always between -1 and 1 .

5. Position is added to meaning · rewatch ▶ 1:29-1:41

5.1 $PE(0) = (0, 1, 0, 1)$; cat at 0 = (0.350, 0.880, 0.610, 1.080). $\sin 0 = 0$ and $\cos 0 = 1$, so only the cosine dimensions change: $-0.120 + 1 = 0.880$, $0.080 + 1 = 1.080$.

5.2 False. E is unchanged. The sum is computed for each input: $x = E[\text{ids}] + PE$.

5.3 B. Same embedding, different PE: (1.259, -0.536 , 0.630, 1.080) vs (0.071, 0.840, 0.670, 1.078).

6. Sinusoidal encoding in code · rewatch ► 1:41-1:54

6.1 2 and 0.02. $10000^{0/4} = 1$ and $10000^{2/4} = 100$, so $PE(2) = (\sin 2, \cos 2, \sin 0.02, \cos 0.02) = (0.909, -0.416, 0.020, 1.000)$.

6.2 About 6.3 and about 628. Pair 1's angle is pos; pair 2's is pos / 100, so it needs 100 times as many positions: the fast and the slow hand.

6.3 (a) [0, 1, 0, 1] and [0.909, -0.416, 0.02, 1.0] · (b) 1.0 · (c) $\approx 7.49, 7.49, 2.70$ (a) Row 0 is $\sin 0 = 0$ and $\cos 0 = 1$ in every pair; row 2 is the $PE(2)$ from the video. (b) Even 10,000 positions in, nothing exceeds 1: no drowning, unlike idea 1. (c) Neighbouring positions score high, far-apart ones lower, and the score depends only on the **distance** (10→11 and 30→31 are equal). Each pair contributes $\sin \cdot \sin + \cos \cdot \cos = \cos(\text{angle difference})$. RoPE, next, builds this “only the gap matters” idea right into attention.

7. RoPE: rotate instead of add · rewatch ► 1:54-2:14

7.1 4 θ and 10 θ ; a gap of 6 θ . Angle = position $\cdot \theta$, and the gap is $(10 - 4)\theta$.

7.2 40° both times. 30° vs 70°, and 1030° vs 1070°: the distance is 4 positions either way.

7.3 B. Sinusoidal encoding adds $PE(\text{pos})$ to the embedding; RoPE turns pairs of numbers in the vectors attention compares.

7.4 False. Only the difference of the angles matters, and that depends only on the distance.